Scope and related work

COGANT in the program-analysis landscape

COGANT sits at the intersection of three established research areas: machine learning for source code, graph-based program representations, and active-inference behavioral modeling. The following subsections place it against each, emphasizing what COGANT provides as infrastructure rather than as a novel model or benchmark.

Machine learning for big code surveys cover naturalness, representation learning, and task taxonomy [@allamanis2018survey]. COGANT contributes **infrastructure**: a documented IR, pipeline, and export contract rather than a new benchmark or model architecture.

Graph neural networks unify message-passing frameworks for relational data [@wu2020comprehensive; @scarselli2009graph]. Although COGANT's primary output is the Active Inference Institute's Generalized Notation Notation, its program graph can be materialised as optional tensor views compatible with these frameworks (PyG, DGL) so that graph-neural-network model

Related tool categories

- **Compiler and LLVM IRs** — richer semantics, heavier toolchain; COGANT favors lightweight extraction for dataset building.
- **SCA and linters** — rule enforcement focus; COGANT focuses on graph generation for downstream learning.
- **Neural code models** (code2vec, CodeBERT, etc.) — often consume tokens or AST paths; COGANT supplies **explicit program graphs** that graph-neural-network-centric research can ingest directly.

Where the full comparison lives

The numbered fragments that follow this file (lexicographic order under Section 8) carry the detailed related-work comparison so tables and proofs are not duplicated here:

- `08_01_landscape_and_tool_categories.md` — landscape overview.
- `08_02_program_analysis_for_ml_and_tables.md` — program analysis for ML, **Table 8** (feature matrix), **Table 9** (input/output contracts), and positioning vs prior art.
- `08_03_lenses_and_synthesis.md` — bidirectional lenses, edit lenses, incremental analysis, categorical framing, and synthesis positioning.
- `08_04_world_models_boundaries_and_compatibility.md` — world models from code, active inference, boundaries, forward compatibility.

Authoritative **implementation scope** (languages, parsers, Rust acceleration) is always